무엇을 어떻게 평가하고, 왜 그렇게 평가하는가

발표 자료를 만드는 팀원을 위한 정리입니다. 파싱·OCR·검색·생성을 어떤 기준으로 쟀는지, 비용을 어떻게 아꼈는지, 최종 평가를 어떤 설정으로 했는지, 질문 화면을 왜 그렇게 만들었는지를 다룹니다. 평가에 쓰는 골든 데이터셋(dev·test)의 분포와 설계 이유는 **보고서 2 · 평가 데이터셋**에 따로 있습니다.

2026-10-19 · 코드 · 설정 · 측정 문서 기준: `rfg_rag` `03ace46` · 과거 측정은 실행마다 입력을 따로 적음 · ⑤ 최종 평가 2026-10-21 · `0d7fe39`

최종 평가 설정을 확정하고 dev 384 · test 880문항으로 최종 평가를 마쳤습니다. test 정답률은 83.7%입니다. ⑤에 확정한 설정과 핵심 결과를, 보고서 3에 지표 전체를 실었습니다. OCR 비교 결과는 dev 점수가 아니라 **방식을 고른 근거**라서 남겼습니다.

먼저 알아야 할 네 가지

1 파싱은 '근거가 끝까지 살아남는가'로 잽니다

글자 유사도가 아니라 정답 근거가 청크와 LLM 문맥까지 온전히 남는지를 셉니다. 지금 팀이 쓰는 v12 입력(18,989청크)에서는 dev 396문항 중 199문항의 근거가 청크까지 입증됐고, 194문항은 추가 확인이 필요합니다.

2 OCR은 후보 실행 전에 써 둔 정답 텍스트로 세 엔진을 비교합니다

표 · 글 · 도식마다 정답 텍스트를 두고 세 엔진을 비교해, 유형마다 채택한 엔진만 전체 320장에 썼습니다. OCR 비용은 모두 $0.49입니다.

3 검색 평가에서 RAGAS를 뺀 가장 큰 이유는 비용입니다

RAGAS 검색 심판은 문항당 $0.00714로 네 평가 심판비의 54%를 차지했습니다. dev 384문항을 한 번 채점하면 약 $2.74이고, 검색 설정을 바꿀 때마다 다시 채점해야 합니다. 그래서 검색은 정해진 공식으로 계산하는 IR 지표, 생성은 핵심 사실 문자열 일치로 바꿔 API 없이($0) 채점하고, 의미 판단이 꼭 필요한 근거 충실도와 최종 정답에만 LLM 심판을 씁니다.

4 화면은 RAG 로직 없이 `answer_question` 하나만 감쌉니다

화면의 답과 평가하는 답이 같은 코드 · 같은 설정에서 나옵니다. 질문 화면 · OCR 라벨링 · OCR 결과 검토를 실행기 하나로 띄웁니다.

평가 지도
무엇을 어디서 · 용어

① 파싱 평가
v12 근거 보존

② OCR 평가
정답 텍스트 · 엔진 비교

③ 검색 · 생성 평가
네 가지로 나눠 채점

④ 비용
아낀 방법과 이유

⑤ 최종 평가
확정 설정 · 결과

⑥ 프런트엔드
세 화면 · 구성 이유

부록 A · B
필요 없음 그림 239장 · 출처

평가 지도

다섯 파이프라인 중 어디를 어떤 자료로 재는지 한 장으로 봅니다. 파란 테두리가 이 보고서의 범위입니다.

① 전처리	② 검색	③ 검증	④ 생성	⑤ 평가
HWP·PDF 파싱, 청킹, OCR	임베딩 + BM25, 기관 라우팅	골든 데이터셋 생성 · 분할 보고서 2	재작성 → 검색 → gpt-6-luna 답변	답변 파일 하나를 네 가지로 채점

파싱 평가	OCR 평가	평가의 자	검색 · 생성 · 근거 · 최종 평가
v12 근거 보존 판정 · 토큰 길이 · 삽입 불변성	정답 텍스트 55장 · 3엔진	dev 384 · test 880문항	Retrieval · Generation 무료 · Grounding · End-to-End 유료

⑥ **프런트엔드** — 질문 화면은 ④의 `answer_question` 을 그대로 부르고, OCR 라벨링 · 검토 화면은 ①의 그림 추출 함수를 다시 씁니다. RAG 로직은 두지 않습니다.

무엇을 어떤 자료로 재나

평가	무엇을 판단하나	기준 자료	지표	비용
파싱	원본의 사실이 청크 · 검색 · 패킹을 지나 LLM 문맥까지 남는가	v12 98문서 · dev 396문항	근거 보존 F · D · B	API 0회
OCR	그림 속 글자 · 표 · 도식을 빠짐없이, 지어내지 않고 옮기는가	후보 출력을 보기 전에 에이전트가 쓴 정답 텍스트 55장	핵심어 재현율 · 정밀도, 편집 유사도	$0.49
Retrieval	검색이 골든 문서 · 청크를 높은 순위로 찾아 문맥에 담았나	골든 dev / test	Recall@k · MRR@k · Evidence Coverage	API 0회
Generation	답에 핵심 사실 문자열이 있나	골든 dev / test	`key_facts_in_answer`	API 0회
Grounding	답이 검색한 청크 안에서만 나왔나 (환각)	골든 dev / test	RAGAS faithfulness	유료 심판
End-to-End	최종 답이 맞나, 얼마나 걸리고 들었나	골든 dev / test	RAGAS factual_correctness, 지연 · 토큰 · 비용	유료 심판

발표에 나올 용어

Recall@k	상위 k개 후보 청크가 정답 문서(또는 청크) 중 몇 %를 담았나. k = 5, 10, 20.
MRR@k	첫 정답이 몇 번째에 나왔나의 역수. 1위면 1, 2위면 0.5, k 안에 없으면 0.
Evidence Coverage	LLM에 실제로 보낸 청크가 정답 청크를 몇 % 담았나. 순위가 아니라 '보낸 것'을 봅니다.
faithfulness	답의 주장이 검색 문맥으로 뒷받침되는 비율 (LLM 심판).
factual_correctness	답과 정답의 사실이 얼마나 겹치나 (LLM 심판).
dev / test	dev는 설정을 고칠 때마다 돌리는 조절용, test는 설정을 고정한 뒤 한 번만 돌리는 최종 점수용. 자세한 것은 보고서 2.

① 파싱은 어떻게 평가하나

파싱 결과에서 질문의 정답 근거가 청크와 LLM 문맥까지 빠짐없이 남는지 봅니다.

한 줄 요약 지금 팀이 쓰는 전처리 입력은 **v12 패키지(98문서 · 18,989청크)**입니다. dev 396문항 중 **199문항은 정답 근거가 청크까지 이어지는 것이 입증**됐고, 194문항은 표 · 조건 · 흩어진 근거라 추가 확인이 필요합니다.

지금 팀이 쓰는 입력 — v12 패키지

경로	`Downloads/Golden_Dev396_v12_Team_Package/` (2026-10-16)
검색 대상	`corpus/chunks.jsonl` 18,989청크 · `corpus/docs.jsonl` 98문서
질문	`data/golden_dev.jsonl` 396문항(30문서). 보고서 2의 dev 408문항에서 답할 수 없음 8 · 틀린 전제 4가 빠진 팀 패키지 구성
코드 기준	`code_reference/` : rfg_rag `1dacd2d` 의 검색 · 색인 · 설정 파일 사본. 리랭커 없음, candidate_k 50 · dense_weight 0.6 · token_budget 2,500
판정 기록	`evidence/summary.json`, `evidence/question_status.json`

근거 보존 판정 — 정답 근거가 v12 청크까지 이어지는가

유형	전체	F 입증	D 추가 확인	B 골든 검토
numeric	104	78	26	0
table	96	87	7	2
condition	69	16	53	0
comparison	56	17	38	1
multi_hop	71	1	70	0
합계	**396**	**199**	**194**	**3**

- **F**: 필요한 핵심 근거가 원문 → 레코드 → 요소 → 청크 경로로 입증된 문항입니다.
- **D**: 표 · 조건 · 여러 곳에 흩어진 근거라 추가 확인이 필요한 문항입니다. 근거가 빠졌다고 판정한 것은 아닙니다.
- **B**: 골든 문항 자체를 다시 볼 후보입니다. 확정 오류가 아니며 정답은 고치지 않았습니다. 확정된 원문 오류는 0건입니다.

이 판정은 청크까지의 보존이며, 검색 · 패킹을 거친 뒤의 전달과 다릅니다. v12 청크로 색인을 새로 만들어 돌린 검색 점수는 ⑤와 보고서 3에 있습니다.

rfg_rag 안에서 한 파싱 점검

rfg_rag로 옮긴 뒤에는 v11 패키지(저장 청크 · 저장 OCR 텍스트)를 재파싱 없이 읽는 경로를 만들고 세 가지를 확인했습니다.

1) 임베딩 길이 — 청크 뒷부분이 잘리지 않는가

입력	문서	청크	KURE 512토큰 초과	2,048토큰 초과	최대 토큰
현재 파서	98	17,614	1,585 (9.0%)	0	1,715
v11 패키지	98	18,984	2,492 (13.1%)	0	858

패키지가 센 o200k 토큰 수를 쓰지 않고, 실제 임베딩 모델(KURE)의 토크나이저로 다시 셌습니다. 512토큰을 넘는 청크가 최대 13%여서 `max_seq_length`를 512 → 2,048로 올렸고, 두 입력 모두 2,048을 넘는 청크는 0개입니다.

2) 그림 설명 삽입 — 위치 · 순서를 지키고, OCR 텍스트가 없으면 기존 출력을 유지하는가

01 실행 조건	문서	전체 청크	설명 청크	확인
저장된 OCR 텍스트 사용	98	22,108	846	391개 위치 · 순서 일치, HWP 표시 없는 80곳은 문서 끝
OCR 텍스트 파일 없음	98	17,614	0	기본 커밋 `58b8b15`의 모든 청크 행과 동일

설명을 넣은 문서는 그림 경계에서 본문을 다시 나누므로, 기존 청크 경계까지 보존하는 것은 아닙니다. API 키를 지우고 네트워크를 막은 채 노트북을 그대로 실행해, OCR 결과를 읽는 경로가 유료 호출을 만들지 않는 것도 확인했습니다.

3) 입력을 바꾸면 골든 라벨부터 다시 붙입니다

골든 근거는 청크 ID가 아니라 본문으로 현재 청크에 다시 연결합니다. 그래서 청킹을 바꾸면 03을 다시 실행해 라벨을 새로 붙이고, 연결률과 제외율을 점수 옆에 함께 보고합니다. 입력별 연결률과 그 한계는 **보고서 2**의 '자의 눈금'에 있습니다.

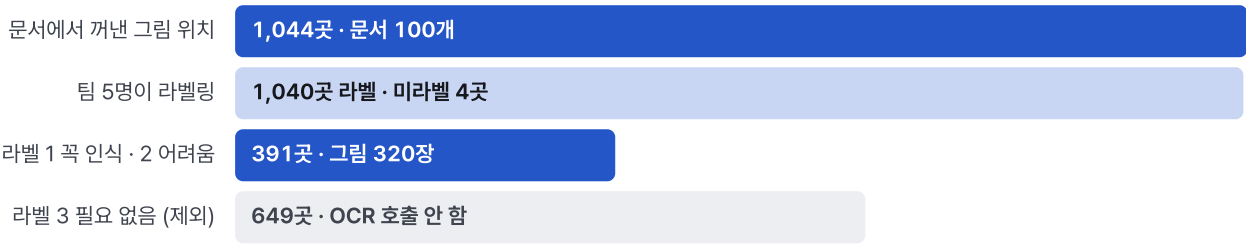
> **한계** v11 대 현재 파서의 네 평가 비교(PR #18)는 질문 16개 중 15개가 연말정산 질문이라 RFP 품질 판단에 쓰지 않았고, 이것이 골든 데이터셋을 새로 만든 이유입니다.

② OCR은 어떻게 평가하나

어떤 그림을 OCR할지 사람이 고르고, 고른 그림을 유형별로 나눈 뒤 같은 정답 텍스트로 세 엔진을 비교합니다.

한 줄 요약 1,044곳의 그림 중 팀이 '필요함'으로 매긴 391곳(서로 다른 그림 320장)만 OCR했습니다. 유형별 정답 텍스트 55장으로 세 엔진을 비교해 **표 · 글은 Gemini, 도식은 GPT**를 골랐고, 빈 출력은 0곳입니다.

어떤 그림을 OCR할지부터 줄였습니다



막대는 모두 그림 위치 수입니다. 라벨링은 ⑥의 OCR 이미지 라벨링 화면에서 묶음 5개로 나눠 했고, 같은 그림이 여러 곳에 나와도 위치마다 따로 매겼습니다. 정답 텍스트 표본은 서로 다른 그림 320장 중 55장(17.2%)입니다. '필요 없음' 649곳이 어떤 그림인지는 다음 쪽에 있습니다.

그림을 유형으로 나누고, 유형마다 기준을 세웠습니다

유형	판단 기준	그림	위치	정답 텍스트
표 table	서식 · 행과 열이 중심	150	160	20
글 text	문장 · 메뉴 · 표지판의 글자가 중심	20	20	15
도식 diagram	연결 · 차트 · 지도 · 구성 관계가 중심	150	211	20

정답 텍스트는 에이전트가 **후보 출력을 보기 전에** 원본 크기로 보고 썼고, 사람의 교정은 받지 않았습니다. 정답 텍스트 파일의 SHA-256은 측정 전에 고정했습니다. 자필을 확실히 읽지 못한 3장은 후보를 부르기 전에 뺐습니다 — 불완전한 기준을 더 잘 읽은 출력과 비교하면 점수가 거꾸로 나오기 때문입니다.

지표와 채택 규칙 — 측정 전에 정했습니다

핵심어 재현율	기준 핵심어 중 출력에 단어 경계까지 맞게 나온 비율. **1순위**.
핵심어 정밀도	출력의 숫자 · 코드 · 단어 중 기준에 있는 비율. 기준에 없는 숫자 · 이름을 지어내면 깎입니다. **2순위**.
편집 유사도	1 − 편집 거리 / 기준 글자 수. 표 문법 차이도 세므로 참고용입니다.
로컬 채택 조건	PaddleOCR 재현율이 최상 API보다 5%p 이내로 낮고 정밀도가 낮지 않으면 로컬을 씁니다. 셋 다 미충족.

'필요 없음'(라벨 3)은 어떤 그림인가

라벨링 화면의 세 선택지는 1 꼭 인식 · 2 인식해야 하나 어려움 · 3 필요 없음입니다. 3은 **OCR로 글자를 꺼내도 질문에 답하는 데 쓸 내용이 없는 그림**에 매겼고, 이 649곳(서로 다른 그림 239장)은 OCR을 부르지 않았습니다. 아래 분류는 이 보고서를 위해 239장을 모두 보고 나눈 것이며, 라벨링 때 따로 매긴 값은 아닙니다.

분류	무엇이 들어 있나	그림	위치
기관 로고 · 슬로건 · 표지 문구	발주기관 · 사업 로고, 캠페인 문구, 직인, 표지의 'Request For Proposal' 같은 장식 글자	129	483
장식 · 아이콘 · QR	구분선 · 막대 · 화살표 · 점 · 테두리 · 모서리 · 단색 상자, 도식에서 잘려 나온 조각, 아이콘, QR 코드	83	120
사진 · 지도	현장 · 건물 · 인물 사진, 위성 사진, 지도 화면	12	12
화면 캡처 · 서식 견본 · 도면	글자는 있지만 팀이 필요 없다고 매긴 것: 기존 시스템 화면, 제출 서식 · 봉투 견본, 배치 도면	15	34
합계		**239**	**649**

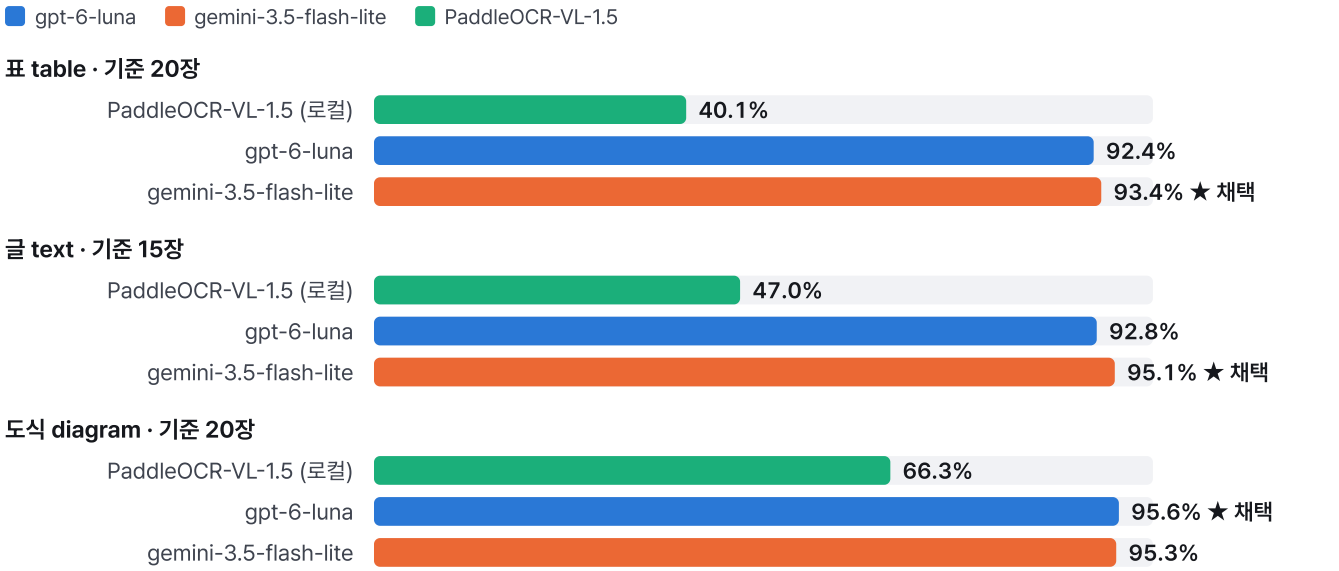
위치 649곳 중 293곳은 고려대학교 로고 한 장이 쪽마다 반복된 것입니다. 로고 · 슬로건만으로 483곳(74%)입니다.

분류별 예시 · 칸 아래 숫자는 그 그림이 나온 위치 수

기관 로고 · 슬로건 · 표지 문구	 293곳	 10곳	 4곳	 1곳	 1곳	 1곳
장식 · 아이콘 · QR	 4곳	 1곳	 2곳	 1곳	 1곳	 1곳
사진 · 지도	 1곳	 1곳	 1곳	 1곳	 1곳	 1곳
화면 캡처 · 서식 견본 · 도면	 1곳	 1곳	 2곳	 1곳	 1곳	 19곳

전체 239장은 부록 A에 분류별로 모두 실었습니다. 그림 원본은 `artifacts/ocr_labels/images/`, 라벨은 `data/ocr_labels/ocr_labels_batch1~5.jsonl`에 있습니다.

유형별 핵심어 재현율 · 같은 기준 55장, 엔진마다 한 번씩



유형	채택	재현율	정밀도	$/이미지	초/이미지	로컬 재현율 격차
표	gemini-3.5-flash-lite	93.4%	85.2%	0.00134	2.8	−53.3%p
글	gemini-3.5-flash-lite	95.1%	80.7%	0.00124	2.9	−48.1%p
도식	gpt-6-luna	95.6%	79.3%	0.00052	10.7	−29.3%p

도식에서 Gemini의 정밀도(85.1%)가 더 높았지만, 미리 정한 1순위가 재현율이라 GPT를 골랐습니다. 로컬 PaddleOCR 시간은 RTX 3080 PC의 측정값입니다.

비용 — 견적을 보이고 승인받은 뒤에만 불렀습니다

단계	호출	사전 견적	실제 비용	메모
기준 55장 비교 (API 두 개)	110	$0.374	$0.105	PaddleOCR는 로컬, 달러 대신 초 기록
나머지 265장 전체 실행	265	$0.249	$0.388	기준 55장 출력은 재사용, 새 출력이 예상보다 길어 +56%
합계	**375**	$0.623	**$0.493**	그림 320장 → 위치 391곳으로 펼침

비용을 줄인 방법

- '필요 없음' 649곳(62%)은 아예 부르지 않았습니다.
- 같은 그림이 여러 곳에 나오면 한 번만 부르고 위치로 펼쳤습니다.
- 기준 55장의 출력은 다시 사지 않았습니다.
- 자동 재시도를 끄고, 저장된 해시 · 엔진 조합은 재호출하지 않습니다.

왜 그렇게 쟀나

- 표 · 글 · 도식은 실패 모양이 달라, 한 엔진의 평균 점수로 고르면 한 유형을 희생합니다.
- 작은 기준 표본으로 먼저 고르면 전체 실행에서 채택하지 않은 엔진에 돈을 쓰지 않습니다.
- 01은 저장된 OCR 텍스트만 읽으므로, 팀원은 API 키 없이 같은 청크를 만듭니다.

한계 기준 55장은 목적 표본이라 320장 전체의 신뢰구간을 말하지 않습니다. OCR 결과를 사람이 확인하는 OCR 결과 검토 화면은 만들었지만 판정은 아직 1/320장입니다. CRAFT 사전 선별(글자 영역 탐지)은 구현만 했고 정확도 · 비용 효과는 검증하지 않았습니다.

③ 검색과 생성은 어떻게 나눠 평가하나

답변 파일 하나를 네 가지로 채점해, 나쁜 답이 어디서 잘못됐는지 가립니다.

한 줄 요약 04가 만든 `answers.jsonl` 하나를 Retrieval · Generation · Grounding · End-to-End가 모두 읽습니다. 앞의 둘은 **API 없이** 계산하고, 의미 판단이 꼭 필요한 뒤의 둘에만 LLM 심판을 씁니다.

answers.jsonl
질문 · 재작성 · 패킹 전 후보 순위 · 실제 보낸 청크 ID · 답 · 토큰 · 비용

→

Retrieval 무료
Recall@k · MRR@k · Evidence Coverage

Generation 무료
핵심 사실 문자열 일치

Grounding 유료
RAGAS faithfulness

End-to-End 유료
factual_correctness · 지연 · 비용

검색 평가에서 RAGAS 대신 점수 기반 지표를 쓴 이유 — 가장 큰 이유는 비용

$0.00714
RAGAS 검색 심판의 문항당 비용. 네 평가 심판비($0.01327)의 54%로 가장 큰 몫

11.7회
검색 평가 한 문항에 부른 심판 호출 수 (16문항에 187회)

$2.74 → $0
dev 384문항 한 번 채점할 때 검색 심판 비용. 검색 설정을 바꿀 때마다 듭니다

평가	이전 (RAGAS 심판)	호출 / 문항	USD / 문항	지금	USD / 문항
Retrieval	context_precision · context_recall	11.7	0.00714	Recall@k · MRR@k · Coverage	**0**
Generation	answer_relevancy	3.0	0.00145	핵심 사실 문자열 일치	**0**
Grounding	faithfulness	2.0	0.00206	그대로 (LLM 심판)	유료
End-to-End	factual_correctness	4.0	0.00262	그대로 (LLM 심판)	유료
합계		20.7	**0.01327**	두 심판만 남김	**0.00468**

같은 단가로 환산하면 Retrieval · Generation 심판만으로 dev 384문항 한 번에 약 $3.30, test 880문항 한 번에 약 $7.56입니다. 검색 설정 비교는 평가 중 가장 자주 돌기 때문에, 이 몫을 $0으로 만든 것이 비용 절감의 핵심입니다.

단가는 이전 16문항 실행의 실제 사용량(`docs/eval_ir_sample.md`)으로 낸 값이며, 질문 길이에 따라 달라지는 추정입니다. 비용 말고도, 골든 근거에 청크 라벨이 생겨 "찾았는지"를 순위로 직접 셀 수 있게 됐고, 같은 입력에는 늘 같은 점수가 나옵니다. LLM 심판은 "맞는 말인가"처럼 문자열로 잴 수 없는 두 곳에만 남겼고, 그 심판 모델 · 프롬프트 · 채점 방식은 바꾸지 않았습니다.

나눈 이유 — 고장 난 곳을 찾으려고

Retrieval	Grounding	End-to-End	진단	다음 할 일
낮음	—	—	**검색 문제**	임베딩 · BM25 · 라우팅 · 가중치 (검색 전용 재실행으로 무료 비교)
높음	낮음	—	**환각**	생성 프롬프트 · 문맥 길이 · 모델
높음	높음	낮음	**정답 · 질문 문제**	골든 정답 · 핵심 사실 · 질문 표현을 다시 봄

한 점수로만 보면 "답이 틀렸다"까지만 압니다. 같은 답변 파일을 네 관점으로 나누면, 검색 담당 · 생성 담당 · 검증 담당 중 누가 고쳐야 하는지가 표의 한 줄로 정해집니다.

검색 평가는 둘로 나눴습니다

A. 순위 — 찾았나

- 중복 제거 · 토큰 예산 패킹 **전** 후보 목록, k = 5 · 10 · 20.
- 문서 · 청크 두 수준. 문서를 먼저 중복 제거하지 않습니다.
- fused · dense만 · BM25만을 한 표로 보고, 요구사항 코드 일치는 순위 없는 집합으로 따로 봅니다.

B. 문맥 — 보냈나

- LLM에 **실제로 보낸** 청크 ID로 Evidence Coverage를 계산합니다.
- 단일 근거와 다중 근거(multi_hop · comparison · 근거 여럿)를 따로 보고합니다.
- MRR은 첫 근거 위치만 보므로 다중 근거의 완전성을 대신하지 않습니다.

왜 순위와 문맥을 나누나. 정답 청크를 20위 안에서 찾아도 토큰 예산 패킹에서 잘리면 LLM은 보지 못합니다. A가 높고 B가 낮으면 검색이 아니라 패킹 · 예산 문제입니다.

채점 규칙 — 점수를 부풀리지 않으려고 정한 것

규칙	내용	왜
일부 라벨 질문은 제외	모든 근거가 청크에 연결된 질문만 청크 지표를 계산합니다. 일부만 연결된 질문의 분모를 줄여 채점하지 않습니다.	쉬운 근거만 남겨 점수가 오르는 것을 막습니다.
제외율을 점수 옆에	(일부 + 미라벨) / 답할 수 있는 질문 수를 주제별로 저장하고 표마다 함께 보입니다.	몇 문항으로 낸 점수인지 모르면 비교할 수 없습니다.
답 없음은 따로	unanswerable과 골든 근거 없는 질문은 IR에서 빼고 각각 셉니다. 답변 문자열 일치는 거절 문장도 채점합니다.	정답 근거가 없는 질문에 Recall을 매길 수 없습니다.
패킹 전 후보로 순위	중복 제거 · 예산 패킹 전 순위로 Recall@k · MRR@k를 셉니다.	검색 순위와 패킹 효과를 섞지 않습니다.
핵심 사실은 정규화한 문자열	공백 · 수치 표기를 정규화하고, 부정문 안의 일치는 맞다고 세지 않습니다. 의미 정답 판정이 아닙니다.	무료로 매번 돌리되, 의미는 factual_correctness가 맡습니다.
불완전한 답변 파일 거부	후보 순위 · 재작성 · 실제 보낸 청크 ID · 검색 설정이 없는 옛 답변은 유료 심판 호출 전에 거부합니다.	유료 호출 전에 채점에 필요한 기록이 모두 있는지 확인합니다.

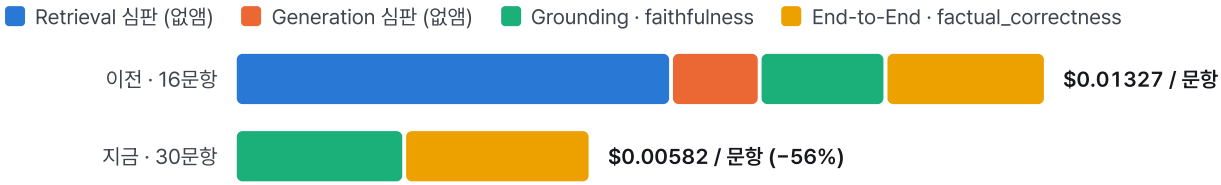
Langfuse 없이도 같은 결과

프로젝트 책임자는 Langfuse Cloud에서 질문별 실행 기록과 그 기록에 연결된 평가 점수를 확인합니다. `answer-question` 기록 안에는 `retrieve-context`와 `generate-answer`가 들어 있습니다. Langfuse 키가 없는 팀원은 05 노트북의 표와 `artifacts/`의 CSV 네 개에서 같은 점수를 확인합니다. 모든 코드 경로가 키 없이 돕니다.

④ 비용을 어떻게 아끼나

OpenAI 키를 팀이 함께 쓰므로, 가장 자주 도는 평가를 무료로 만들고 유료 호출은 값을 먼저 보입니다.

문항당 심판 비용 · 이전 16문항 실행과 dev 30문항 표본 실행, 같은 축



비용 비교만 싣고 점수는 싣지 않습니다. 질문 · 시점이 다른 두 실행이라 통제 실험은 아니며, 줄어든 몫의 대부분은 없앤 Retrieval · Generation 심판입니다.

방법	무엇을 했나	효과
심판을 지표로	Retrieval · Generation의 LLM 심판(이전 235회 호출)을 IR 지표 · 문자열 일치로 교체	두 평가 **$0**, 같은 입력엔 늘 같은 점수
검색 전용 재실행	저장된 재작성 질문으로 검색만 다시 돌림. 재작성 · 생성 · 심판 호출 없음	검색 설정 비교 **$0**, 사용량 CSV 바이트 단위 동일
표본으로 제한	유료 확인은 모든 주제를 포함한 dev 30문항 표본만	한 번에 240회 호출, **$0.24**
견적 → 승인 → 실측	실행 전 config 단가로 견적을 보이고 승인 뒤 실행, 실제 비용과 비교	견적 $0.19 · 실제 $0.24 (+25%) — 차이도 기록
test는 한 번	설정을 고정한 뒤 test 880문항을 한 번만 평가 (2026-10-21)	test로 설정을 고르지 않아 누수와 반복 비용을 함께 막음
모든 호출 기록	OCR · 재작성 · 답변 · 심판 호출마다 `api_usage.csv` 에 한 행, 재시도도 포함	Langfuse 키 없는 팀원도 비용을 봄

dev 30문항 표본 한 번의 비용

04 재작성 · 30회	$0.0166
04 답변 · 30회	$0.0471
05 faithfulness · 60회	$0.0832
05 factual_correctness · 120회	$0.0915
합계 · 240회	**$0.2384**

왜 이렇게 했나

- 모든 유료 호출은 먼저 값을 보이고 승인받습니다.
- 검색 설정은 자주 바뀌므로, 가장 자주 도는 평가를 무료로 만들어야 실험을 많이 할 수 있습니다.
- 심판은 문항마다 여러 번 부르므로(faithfulness 2회, factual_correctness 4회) 문항 수를 줄이는 효과가 큽니다.

⑤ 최종 평가 — 확정한 설정과 결과

dev로 청킹 · 필터를 정하고, 그 값을 고정한 뒤 test를 한 번 돌렸습니다. 지표 전체와 해석은 보고서 3에 있습니다.

1. 청킹 확정
구조 기반 v12 · 길이 기반 700자 비교
완료

→

2. 검색 설정 확정
dev 검색 비교 8조합 ($0)
완료

→

3. dev 유료 확인
dev 384문항, 견적 승인 후
완료

→

4. 고정 · test 1회
test 880문항, 네 평가 한 번
완료 · 2026-10-21

83.7%
test 정답률 · 답할 수 있는 872문항 (dev 85.4%)

0.915
test Evidence Coverage · 정답 근거 청크가 답변 모델에 간 몫

5.0 · 6.9초
test 전체 응답 시간 p50 · p90

$8.76
dev · test 생성과 심판을 합친 실제 이용료

확정한 값

항목	확정값	dev에서 본 근거
청킹	구조 기반 v12 청크 · 98문서 · 18,989청크	길이 기반 700자와 검색 지표가 비슷함(dev 267문항, Chunk MRR@10 0.785 대 0.768). 팀 합의로 구조 기반
기관 필터	켬	끄면 구조 기반의 Chunk Recall@10이 0.955 → 0.502, 비슷한 RFP끼리 섞임
수량 필터	켬	켜도 꺼도 같음. 이 필터가 쓰일 질문이 dev · test에 없어 효과는 재지 못함
임베딩 · BM25	`nlpai-lab/KURE-v1` 2,048토큰 · Kiwi 형태소	로컬 7개 비교에서 Nemotron이 R@10 78.4% 대 74.7%였으나 VRAM · 색인이 더 큼. 최종 실행은 KURE-v1
융합 · 예산	RRF · `candidate_k` 50 · `token_budget` 2,500	리랭커 없음
모델	재작성 · 답변 gpt-6-luna, 심판 gpt-5-mini (추론 low)	—

평가 문항은 골든 dev 408 · test 943문항에서 핵심 사실을 뽑을 수 없는 다중 근거 문항(dev 24 · test 63)을 뺀 dev 384 · test 880문항입니다. 답할 수 있는 질문은 답 없음 8문항씩을 뺀 376 · 872문항입니다.

남긴 기록 — `최종평가_결과/` 폴더

설정 · 코드	`재현/config.yaml`, `재현/코드_기준.md` — 커밋 `0d7fe39`, 게이트 1,693 통과
결과	`지표/` 표 CSV, `그래프/` 그림, `문항별/` 질문마다 점수와 답변
비용	`재현/api_usage.csv` — 생성 dev $0.27 · test $0.63, 심판 dev $2.33 · test $5.52
test 뒤 금지	test 결과를 보고 설정을 바꾸지 않습니다. 바꾸면 그 test는 더는 최종 점수가 아닙니다.

문항별 결과에는 실제 RFP 질문과 답변이 들어 있어 Git에 올리지 않습니다.

⑥ 프런트엔드는 어떻게 구성했나

세 화면, 서버 둘, 실행기 하나. 화면은 RAG 로직을 갖지 않습니다.

질문 화면
index.html · RFP를 검토하는 팀원
무엇을 RFP에 대해 묻고 근거 문단을 바로 확인
성공 답의 [n]을 누르면 옆에 원문 청크가 열림

OCR 이미지 라벨링
ocr.html · OCR 대상을 고르는 팀원 5명
무엇을 그림 위치마다 1 꼭 · 2 어려움 · 3 필요 없음
성공 누르는 즉시 저장, 다시 열어도 유지

OCR 결과 검토
review.html · OCR 결과를 확인하는 팀원
무엇을 원본과 저장된 OCR 텍스트를 나란히 보고 판정
성공 OK · 일부 틀림 · 틀림이 OCR 텍스트 해시에 묶여 저장

구성

브라우저
React 19 + Vite
localhost:5173
세 HTML 진입점

→ **`/api` → server.py**
FastAPI · 포트 8765
`POST /api/ask` 하나 (53줄)

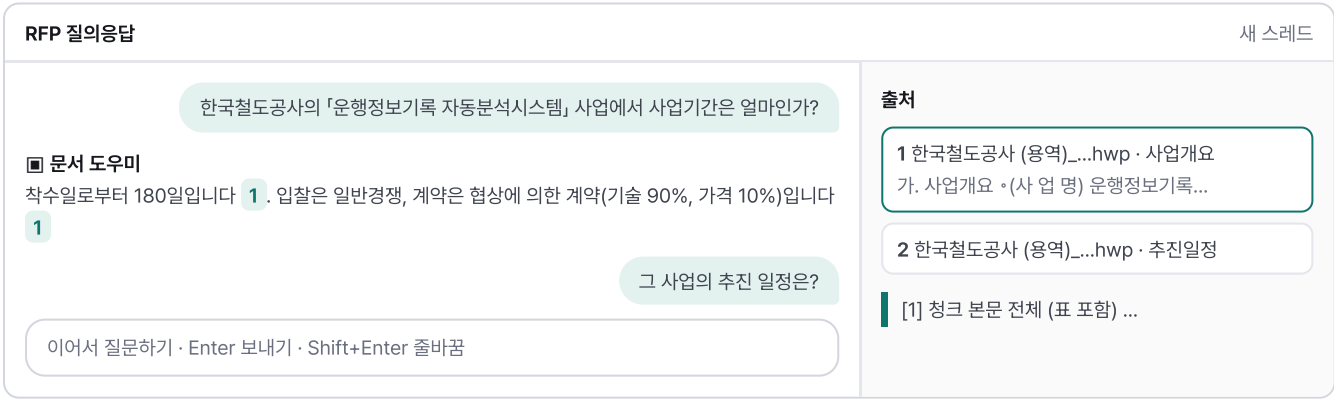
→ **rag/ 의 answer_question**
재작성 → 하이브리드 검색 → 답변. 04 · 평가와 같은 함수 · 같은 설정

↘ **`/ocr-api` → ocr_select/server.py**
FastAPI · 포트 8766
그림 · 라벨 · 판정 · 원본 보기

→ **artifacts/ocr_labels/**
`labels.jsonl` · `reviews.jsonl`
덧붙여 쓰기

`frontend/start.cmd` (macOS는 `start.command`)를 더블클릭하면 창 하나에서 세 서버가 함께 뜨고 브라우저가 열립니다. 창을 닫으면 모두 꺼집니다. 화면은 React와 React DOM으로 만들고, Markdown · 표 표시는 react-markdown과 remark-gfm, 글꼴은 Pretendard를 씁니다. 개발 · 빌드 도구는 Vite입니다.

질문 화면



① 오른쪽 말풍선 = 나, 왼쪽 이름표 = 문서 도우미. 답은 말풍선에 넣지 않아 긴 표가 갇히지 않습니다.

② 출처 칸은 답 옆에 고정. 근거를 보려고 스크롤하지 않습니다. 1024px보다 좁으면 답 위로 접힙니다.

③ 이어서 물으면 이 탭의 최근 10턴을 함께 보냅니다. 질문을 재작성할 때 대화 기록을 보고 "그 사업"이 가리키는 사업명을 채웁니다. 서버는 대화를 저장하지 않습니다.

배치를 보이기 위한 그림입니다. 질문과 답 내용은 dev 골든 질문과 그 근거 문단에서 옮겼으며, 실제 화면 응답을 캡처한 것은 아닙니다.

왜 이렇게 만들었나

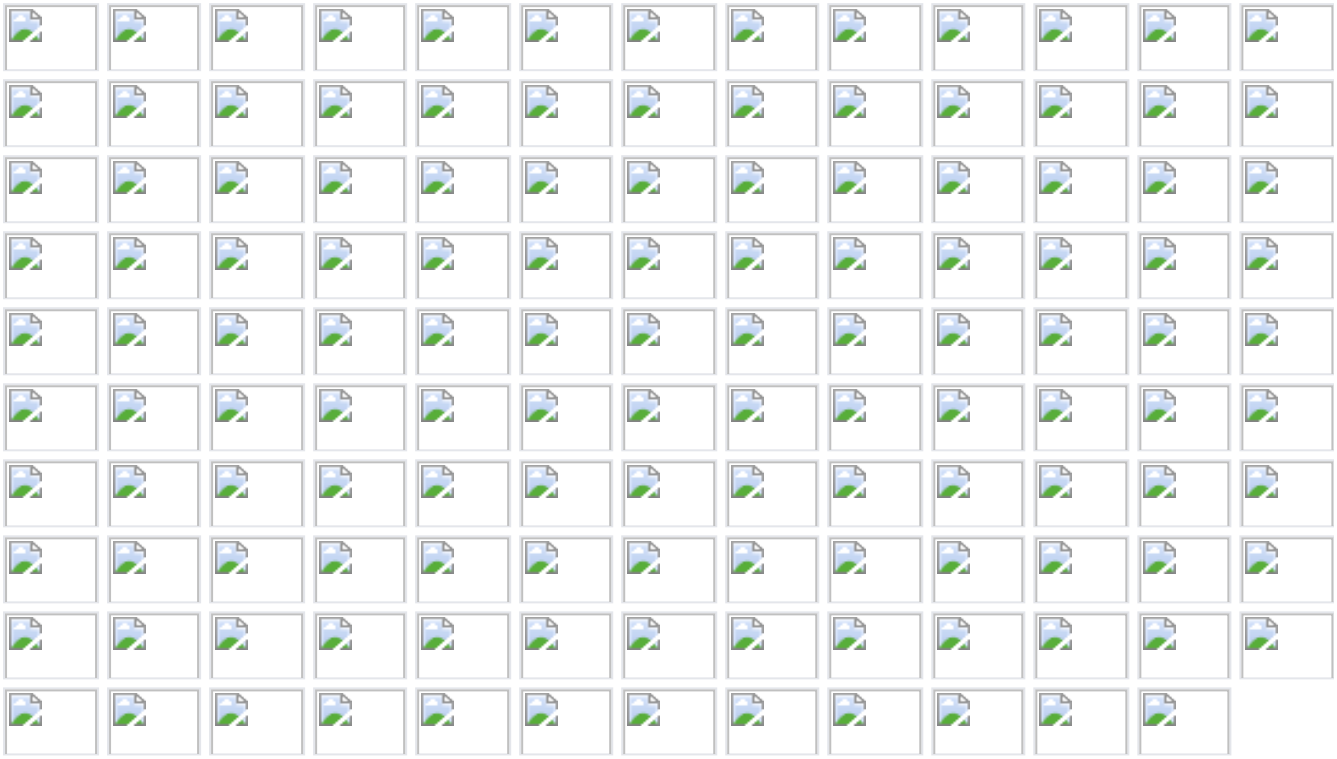
결정	이유
RAG 로직을 두지 않음	화면이 `answer_question` 만 부르므로, 화면의 답이 04 · 평가에서 채점한 답과 같은 코드 · 같은 설정에서 나옵니다. 화면에서 고친 것이 평가를 비켜 가지 않습니다.
채팅 형태	질문을 제목처럼 크게 쓴 첫 판에 "입력 · 출력의 채팅 직관성이 떨어진다"는 피드백. 누가 말했는지를 위치로 바로 보이게 했습니다.
출처를 답 옆 칸에	답 위 카드 격자였던 첫 판은 넓은 화면 양옆을 비우고 스크롤을 요구해 "모바일 같다"는 피드백. 근거 확인이 이 화면의 성공 기준입니다.
카드 두 줄 (파일 · 쪽 / 첫머리)	한 질문의 출처가 같은 파일인 경우가 많고 HWP는 쪽 번호가 없어, 파일 이름만으로는 카드를 구별할 수 없습니다.
글자 최소 15px, 단계 넷	13px이 "흐리고 작다"는 피드백. display · body · label · meta 네 단계만 쓰고, Pretendard로 OS마다 같은 모양을 냅니다.
강조색은 근거로 가는 길에만	청록을 보내기 · [n] 칩 · 펼친 카드에만 써서 눈이 본문 → 칩 → 카드로 따라갑니다. 파랑은 링크와 겹쳐 뺐습니다.
라벨링: 위치마다 카드	서로 다른 그림마다 카드 하나였던 첫 판(505장)은 "그림이 줄었다"는 피드백. 같은 로고도 나온 곳마다 따로 매깁니다.
라벨링: 묶음 5개 · 내보내기 · 합치기	팀원 다섯이 나눠 매기도록 카드를 문서 · 위치 순으로 다섯 등분(209 · 209 · 209 · 209 · 208곳). 누구 PC에서 추출해도 같은 묶음입니다.
라벨링: 크게 보기 · 원본 보기	220px 미리보기로는 글자 유무를 판단하기 어렵습니다. PDF는 그 쪽으로, 쪽 번호가 없는 HWP는 그림 앞뒤 본문을 띄웁니다.
검토: 판정을 OCR 텍스트 해시에 묶음	OCR 텍스트 · 엔진 · 유형이 바뀌면 이전 판정을 '미검토'로 돌려, 옛 텍스트에 대한 OK가 새 텍스트의 OK로 남지 않게 합니다.
실행기 하나	팀원이 터미널 세 개를 띄우지 않아도 되도록 더블클릭 한 번. 창을 닫으면 서버가 남지 않습니다.

사용량 화면에서 묻는 질문도 OpenAI를 부르므로 `api_usage.csv` 에 재작성 · 답변 행이 쌓입니다. 평가 점수 CSV에는 영향이 없습니다.

부록 A · '필요 없음' 그림 239장 전체

라벨 3을 받은 서로 다른 그림을 ②의 분류대로, 나온 위치가 많은 순서로 모았습니다.

기관 로고 · 슬로건 · 표지 문구 · 그림 129장 · 위치 483곳



장식 · 아이콘 · QR · 그림 83장 · 위치 120곳

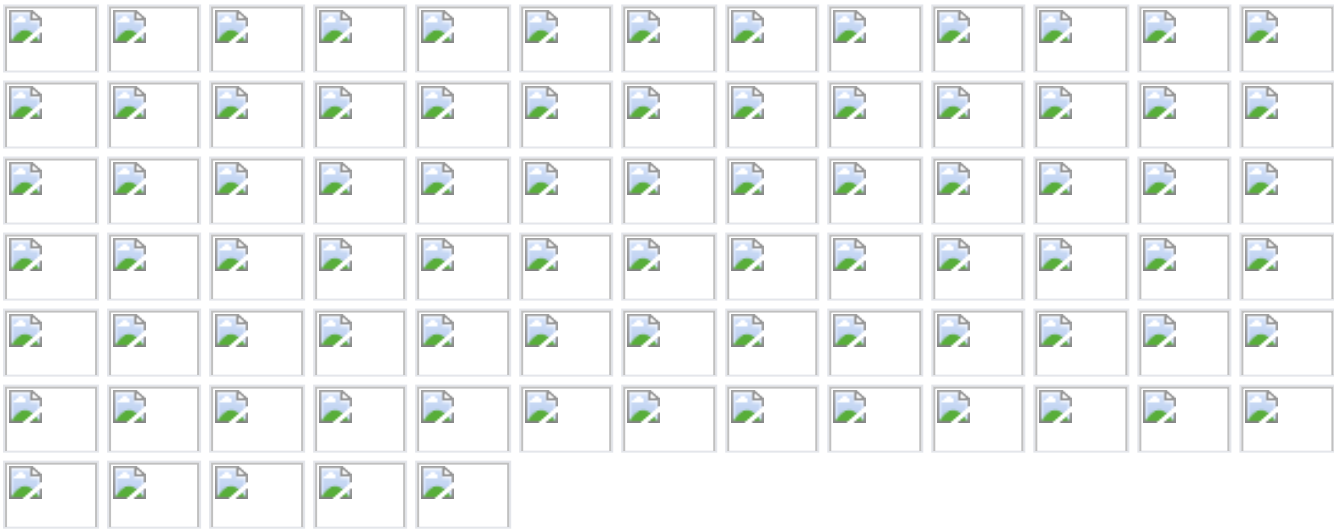
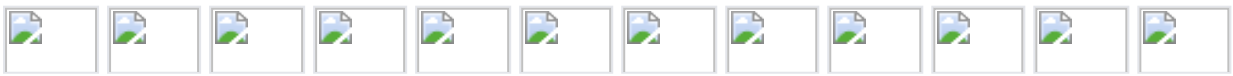
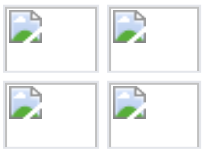


사진 · 지도 · 그림 12장 · 위치 12곳



화면 캡처 · 서식 견본 · 도면 · 그림 15장 · 위치 34곳



부록 B · 출처

이 보고서의 수치를 옮겨 온 곳입니다. 발표 자료에 숫자를 인용할 때 이 파일을 함께 표기하세요.

절	파일	내용
① 파싱	`Downloads/Golden_Dev396_v12_Team_Package/README.md`, `…/evidence/summary.json`	v12 입력 · dev 396 · F/D/B 판정
	PR #18, `docs/ocr_benchmark.md` 끝 절	KURE 토큰 길이, 삽입 불변성
② OCR	`docs/ocr_benchmark.md`, `바탕화면/RFP_OCR_파싱/분포.md`	라벨 집계, 정답 텍스트, 엔진 비교, 비용
	`data/ocr_labels/ocr_labels_batch1~5.jsonl`, `artifacts/ocr_labels/images/`	'필요 없음' 649곳 · 239장, 분류는 이 보고서에서 그림을 보고 나눔
③ 평가	`README.md` 평가 절, `rag/evaluation/`	네 평가 정의, IR 공식, 제외 규칙
④ 비용	`docs/eval_ir_sample.md`	견적 · 실제 비용, 심판 비용, 검색 전용 재실행
⑤ 최종	`최종평가_결과/` (`재현/config.yaml` · `재현/코드_기준.md` · `지표/`), 보고서 3, `docs/benchmarks/seven_model/full_dev/report.md`	확정 설정, 결과 지표, 후보 근거
⑥ 화면	`frontend/DESIGN.md`, `frontend/` 코드	화면 목적, 배치 이유, 피드백